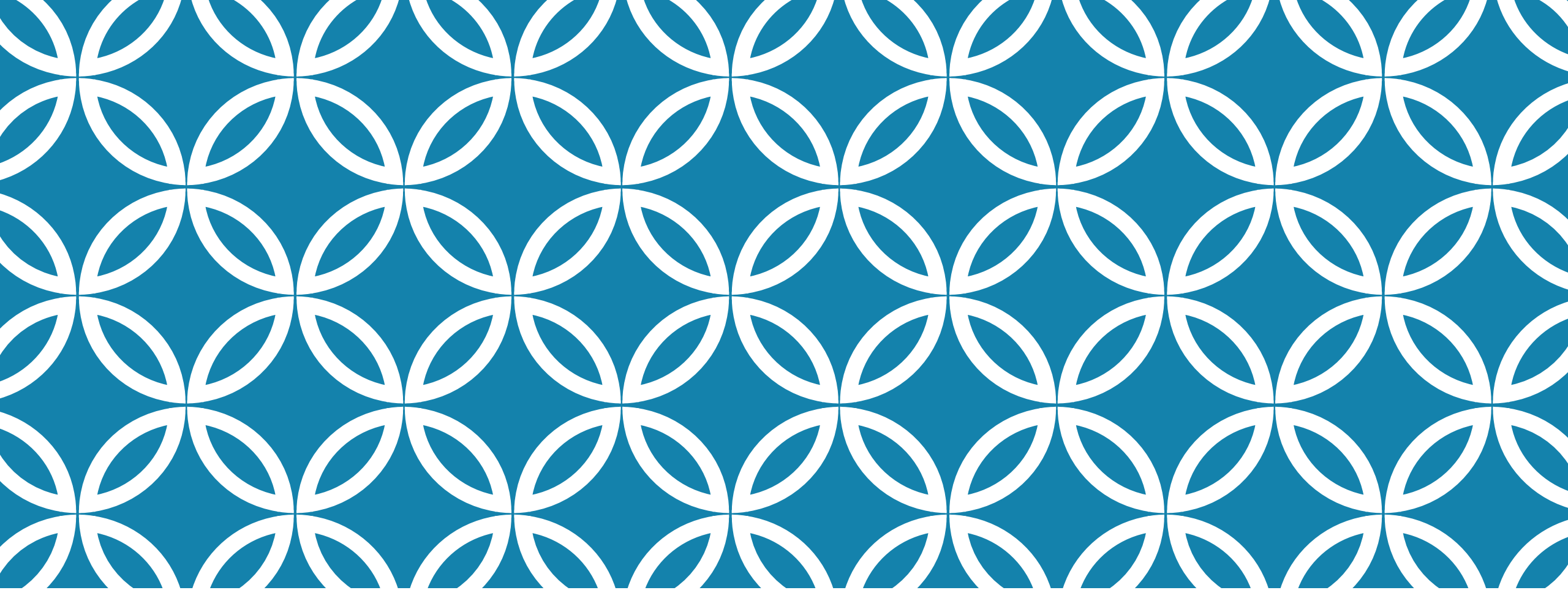


LR PARSERS

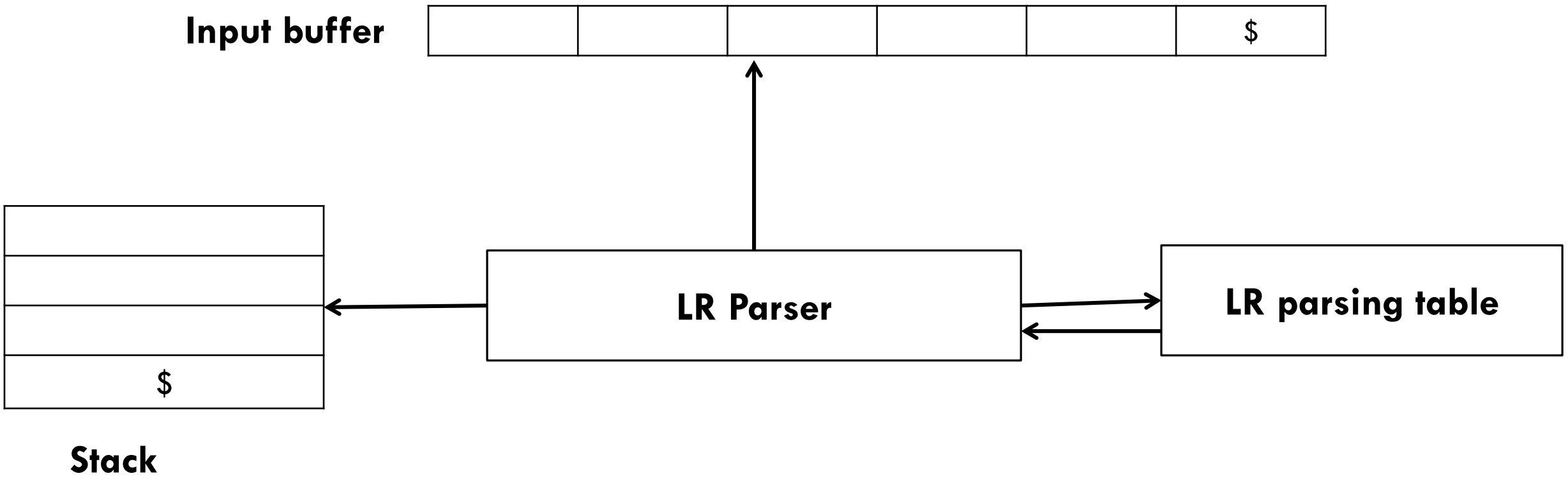
LR PARSER

Bottom up parser

'L' → input is read from left to right

'R' → produces a rightmost derivation

Also called LR(0) parser


Input buffer
$
LR Parser
LR parsing table
$
Stack

Parsing table consists of two parts

- ACTION
- GOTO

To construct LR(0) parsing table, we use the canonical collection of LR(0) items

STEPS

1. Augment the grammar
2. Number the productions
3. Draw the canonical collection of LR(0) items – data flow diagram (DFA)
4. Create the parsing table
5. Perform stack implementation
6. Draw parse tree

EXAMPLE – 1

S → XX

X → aX | b

Check if it can be parsed by LR(0) parser

If yes, then parse string “**aabb**$”

(1) Augment the grammar

S’ → S

S → XX

X → aX

X → b

(2) Number the production

0: S’ → S

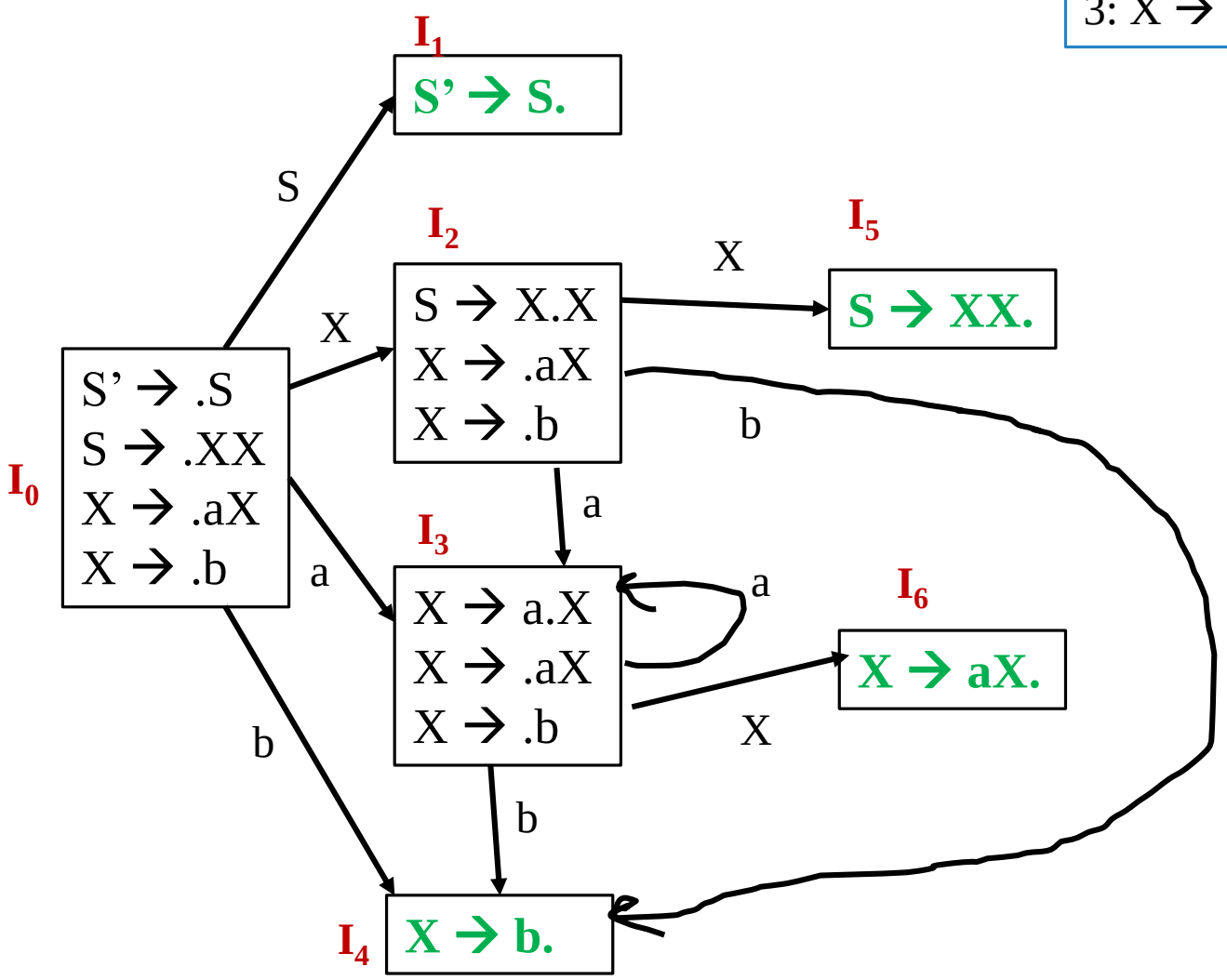
1: S → XX

2: X → aX

3: X → b

(3) Find the LR(0) canonical items

0: S' → S
1: S → XX
2: X → aX
3: X → b



(4) Create the LR(0) Parser table

States	Action			Goto	
	a	b	$	X	S
I_0	S_3	S_4		2	1
I_1			Accept		
I_2	S_3	S_4		5	
I_3	S_3	S_4		6	
I_4	r_3	r_3	r_3		
I_5	r_1	r_1	r_1		
I_6	r_2	r_2	r_2		

No Shift-reduce or Reduce-reduce conflict
Hence grammar can be parsed using LR(0) parser

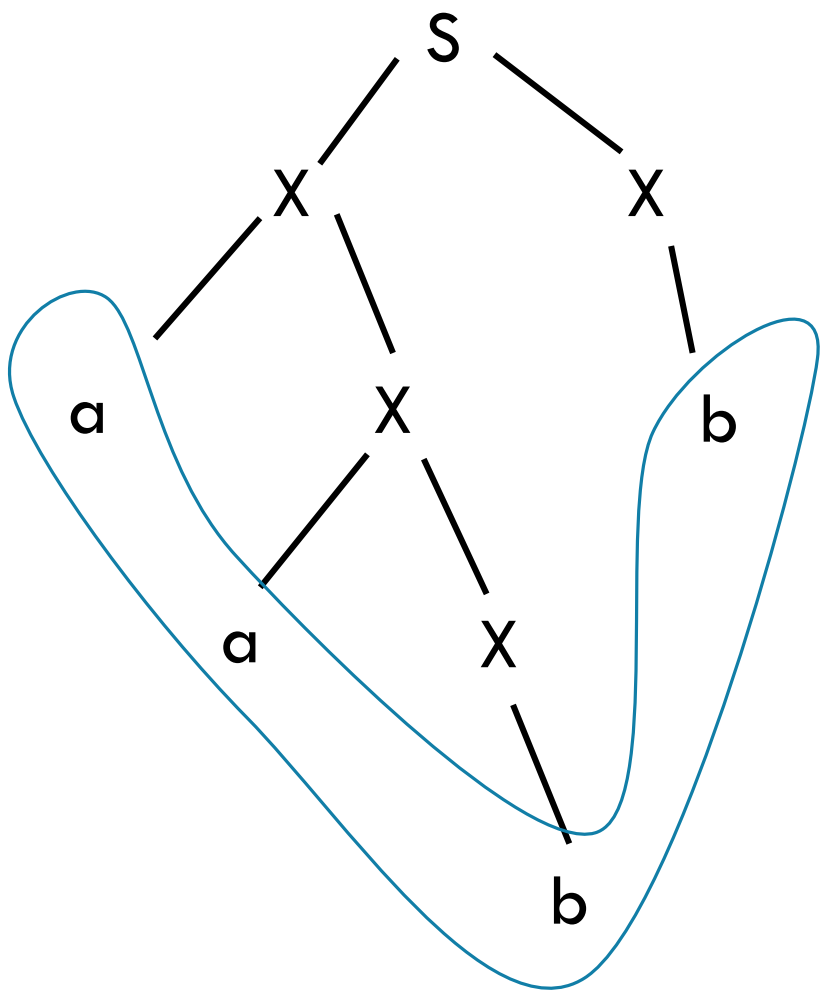
(5) Stack implementation of input string **aabb**$

Stack	Input string	Action
$**0**	**a**abb$	Shift a3
$0a**3**	**a**bb$	Shift a3
$0a3a**3**	**b**b$	Shift b4
$0a3a3b**4**	**b**$	Reduce X → b
$0a3a**3X6**	**b**$	Reduce X → aX
$0a**3X6**	**b**$	Reduce X → aX
$**0X2**	**b**$	Shift b4
$0X2b**4**	**$**	Reduce X → b
$0X**2X5**	$	Reduce S → XX
$**0S1**	**$**	Accept

States	Action			Goto	
	a	b	$	X	S
I_0	S_3	S_4		2	1
I_1			Accept		
I_2	S_3	S_4		5	
I_3	S_3	S_4		6	
I_4	r_3	r_3	r_3		
I_5	r_1	r_1	r_1		
I_6	r_2	r_2	r_2		

(6) Draw parse tree of input string **aabb**$

Start reading the stack table from the bottom



Stack	**Action**
$**0**	Shift a3
$0a**3**	Shift a3
$0a3a**3**	Shift b4
$0a3a3b**4**	Reduce X → b
$0a3a**3X6**	Reduce X → aX
$0a**3X6**	Reduce X → aX
$**0X2**	Shift b4
$0X2b**4**	Reduce X → b
$0X**2X5**	Reduce S → XX
$**0S1**	Accept

EXAMPLE – 2

A → (A) | a

Check if it can be parsed by LR(0) parser

If yes, then parse string “**((a))**$”

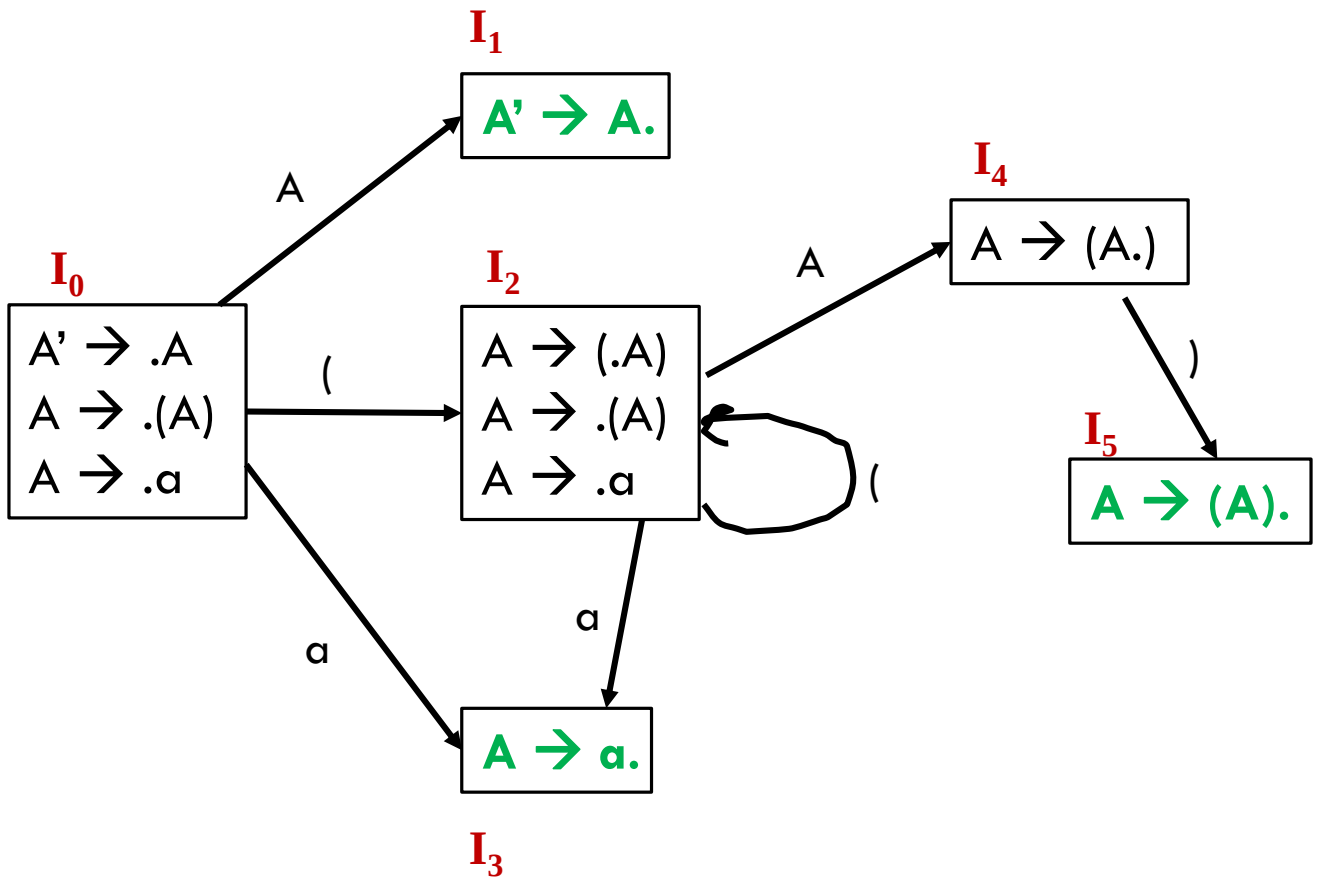
(1) Augment and (2) number the productions

0: A' → A

1: A → (A)

2: A → a

(3) Create the LR(0) canonical items



(4) Create the LR(0) Parser table

States	Action				Goto
	a	(	)	$	A
I_0	S_3	S_2			1
I_1				accept	
I_2	S_3	S_2			4
I_3	R_2	R_2	R_2	R_2	
I_4			S_5		
I_5	R_1	R_1	R_1	R_1	

No Shift-reduce or Reduce-reduce conflict

Hence grammar can be parsed using LR(0) parser

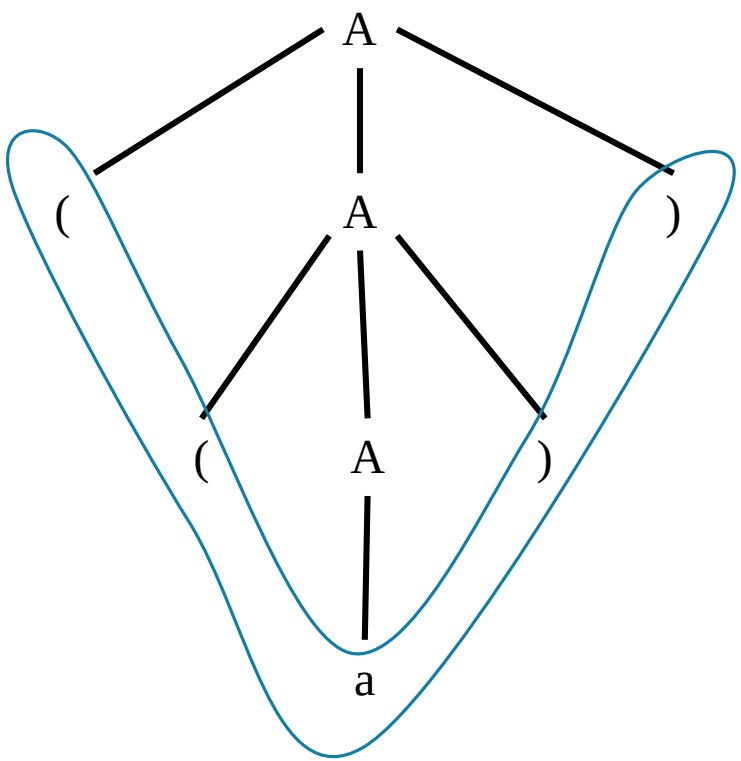
(5) Stack implementation of input string "((a))$"

Stack	Input string	Action
$0	((a))$	Shift (2
$0(2	(a))$	Shift (2
$0(2(2	a))$	Shift a3
$0(2(2a3	))$	Reduce A → a
$0(2(2A4	))$	Shift)5
$0(2(2A4)5	)$	Reduce A → (A)
$0(2A4	)$	Shift)5
$0(2A4)5	$	Reduce A → (A)
$0A1	$	Accept

States	Action				Goto
	a	(	)	$	A
I_0	S_3	S_2			1
I_1				accept	
I_2	S_3	S_2			4
I_3	R_2	R_2	R_2	R_2	
I_4			S_5		
I_5	R_1	R_1	R_1	R_1	

0: A' → A
1: A → (A)
2: A → a

(6) Parse tree of input string “**((a))$**”



Stack	**Input string**	**Action**
$0	((a))$	Shift (2
$0(2	(a))$	Shift (2
$0(2(2	a))$	Shift a3
$0(2(2a3	))$	Reduce A → a
$0(2(2A4	))$	Shift)5
$0(2(2A4)5	)$	Reduce A → (A)
$0(2A4	)$	Shift)5
$0(2A4)5	$	Reduce A → (A)
$0A1	$	Accept

EXAMPLE – 3

E → T + E | T

T → id

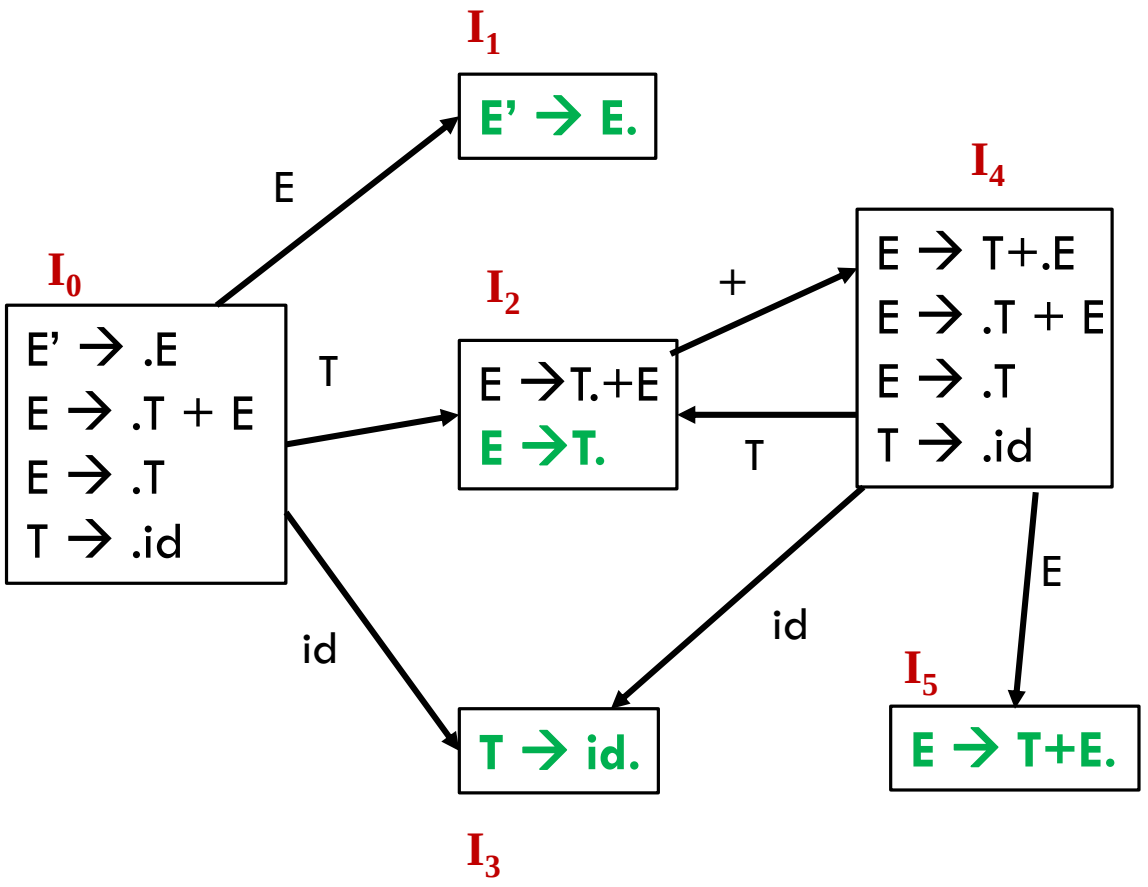
Check if the grammar can be parsed by LR(0) parser

If yes, then parse string "**id+id**$"

(1) Augment and (2) number the productions

0: E' → E
1: E → T + E
2: E → T
3: T → id

(3) Create the LR(0) canonical items



(4) Create the LR(0) Parser table

States	Action			Goto	
	+	id	$	E	T
I_0		S_3		1	2
I_1			accept		
I_2	S_4 / R_2	R_2	R_2		
I_3	R_3	R_3	R_3		
I_4		S_3		5	2
I_5	R_1	R_1	R_1		

Since there is a Shift-reduce conflict
The grammar cannot be parsed using LR(0) parser

EXERCISES

(1) S → Aa | bAc | bda

A → d

(2) S → L = R | R

L → *R | id

R → L

EXERCISES

(3) S → AS | b

A → SA | a

(4) S → aZ | b

Z → a | bS

(5) S → aSb | ε